Philippine languages cost 1.8–2× more tokens than English across six open-model tokenizers

PinoAI research note · EXP-TOK-001

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-09-29

Abstract

We measure how efficiently the tokenizers of five candidate base models, and one reference family, encode Philippine languages. On 1,012 aligned FLORES+ devtest sentences, every tokenizer spends 1.77–2.04× as many tokens on Filipino, Cebuano, Ilocano and Waray as on the same English content. Pangasinan costs 1.45–1.54×. English costs 1.24–1.27 tokens per word for every tokenizer, so the gap is specific to Philippine languages. Gemma 4′s tokenizer is the most efficient on every Philippine language in all three text sets. On Filipino it uses 6.8% fewer tokens than Qwen3.5 and 12.3% fewer than Sailor2. This supports the pre-registered H-DEC-001 Part B. The result was independently reproduced with 0 differences across 876 values. We recommend keeping each model's original tokenizer for the base model decision (DEC-001), and testing vocabulary extension only once a Philippine corpus exists.

1. Introduction

Training compute and context length are counted in tokens. A language that needs more tokens per idea costs more to train on, and fits less into the context window. Prior work shows that multilingual tokenizers split non-English text more finely [1], and that the same content can cost several times more tokens across languages [2]. No published measurement covers PinoAI's candidate models on Philippine languages. This study answers that question before we choose a base model (DEC-001), and before we decide whether a Philippine tokenizer (EXP-TOK-002/003) is worth its cost.

2. Pre-registered hypothesis

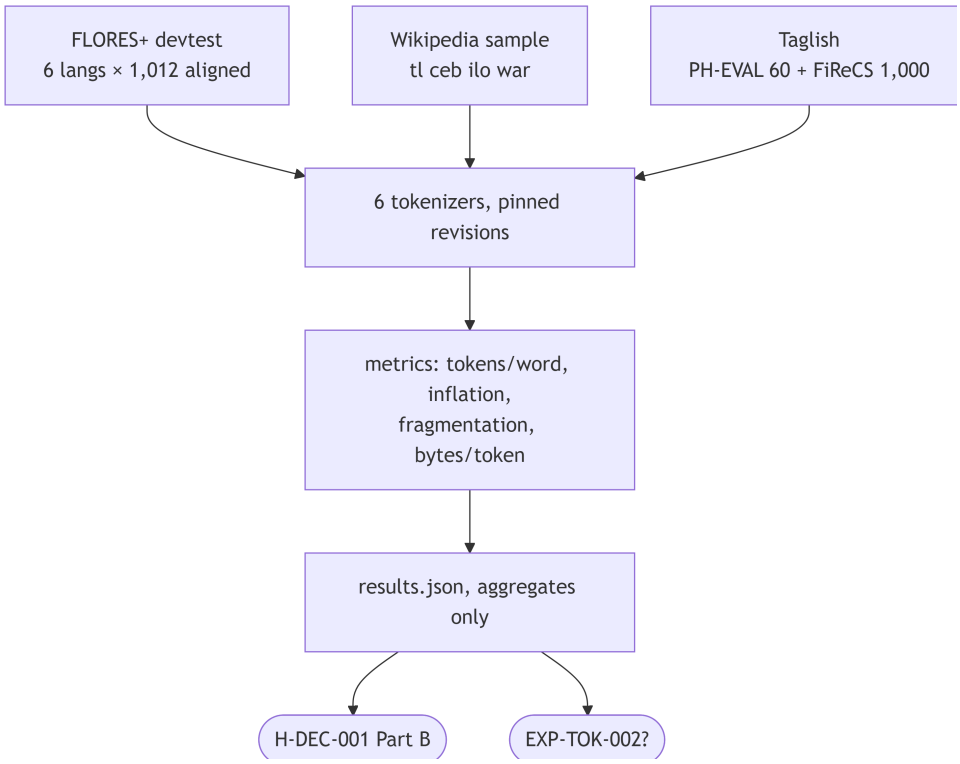
H-DEC-001, stated by the project lead on 2026-09-28 and recorded in the project decision log at **2026-09-29T04:20Z**, before any result. Part B: *"Gemma 4 12B … has lower Filipino token fertility (EXP-TOK-001)."* Operationalized in advance as `tokens_per_word` on FLORES+ devtest `fil_Latn`, gemma-4-12b vs qwen3.5-9b-base and sailor2-8b. Lower is better.

3. Method

- **Tokenizers** (only tokenizers were loaded, no weights), at the revisions pinned in `experiments/configs/EXP-TOK-001.yaml`:
  - Qwen3.5 (`Qwen/Qwen3.5-9B-Base` rev. 68c46c4b34)
  - Gemma 4 (`google/gemma-4-12B` rev. 023679ed35)
  - Sailor2 (`sail/Sailor2-8B` rev. fb9e3b997b)
  - Mistral Tekken (`mistralai/Ministral-3-8B-Base-2512` rev. d4883f9b36)
  - Apertus (`aisingapore/Apertus-SEA-LION-v4-8B-IT` rev. 3cb0202547)
  - a Llama-3-style tokenizer (`HuggingFaceTB/SmolLM3-3B-Base` rev. d78a42f791)
- **Data:**
  - FLORES+ devtest [3] (`openlanguagedata/flores_plus` rev. 5fec6c13f9): 1,012 aligned sentences each for eng, fil, ceb, ilo, war and pag. Alignment is asserted by row id.
  - Wikipedia 20231101: 2,000 paragraphs each for tl, ceb, ilo and war (first parquet shard, seed 20260928).
  - Taglish: 60 native PH-EVAL prompts and 1,000 FiReCS test reviews.
- **Metrics:**
  - tokens per whitespace word;
  - inflation = tokens(lang) / tokens(eng) over aligned sentences (total ratio and per-sentence median);

- share of words split into ≥3 tokens;
- UTF-8 bytes per token.

- **Command:** `python -m phfm.models.tokenizers.measure --out experiments/reports/EXP-TOK-001` (`src/phfm/models/tokenizers/measure.py`).
- **Compute:** CPU only, 0 GPU hours. Python 3.14; transformers 5.17.0, tokenizers 0.23.2, huggingface_hub 1.33.0. Run 2026-09-29 around 05:02Z.
- **Statistics:** tokenization is deterministic, so no sampling test applies to the FLORES+ comparison. Uncertainty comes from which text was sampled, and is discussed in §6.
- **Governance:** FLORES+ and FiReCS are cleared for measurement (internal data-governance register). Only aggregates are stored; no source sentence is.



4. Results

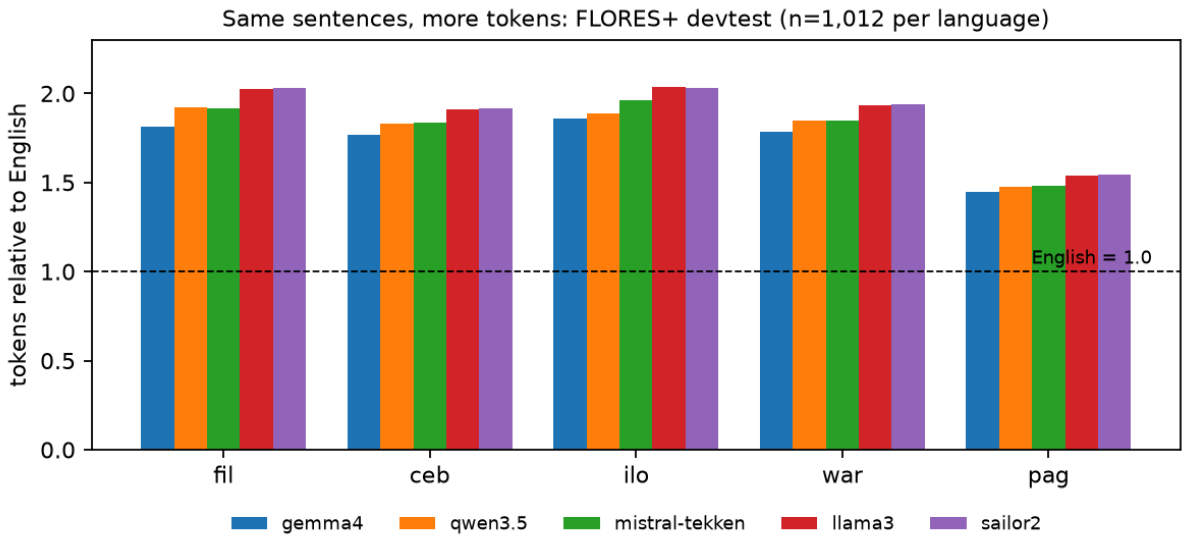


Figure 1: Inflation vs English on aligned FLORES+ sentences. Apertus is omitted because it equals Mistral Tekken on every value. Source: `sequence_inflation_vs_english` in results.json.

Table 1: Tokens per word, FLORES+ devtest. Source: `sets.flores_plus_devtest.<tok>.<lang>.tokens_per_word`; vocab = `len_tokenizer`.

tokenizer (vocab)	eng	fil	ceb	ilo	war	pag
gemma4 (262,144)	1.238	1.840	1.826	1.943	1.770	1.860
qwen3.5 (248,077)	1.257	1.974	1.915	2.002	1.858	1.921
mistral-tekken (131,072)	1.271	1.992	1.947	2.102	1.878	1.952
apertus (131,072)	1.271	1.992	1.947	2.102	1.878	1.952
llama3 (128,256)	1.241	2.056	1.974	2.133	1.918	1.981
sailor2 (151,665)	1.261	2.098	2.013	2.162	1.955	2.019

Table 2: Inflation vs English (total / per-sentence median). Source: `sequence_inflation_vs_english` / `sequence_inflation_median`.

tokenizer	fil	ceb	ilo	war	pag
gemma4	1.82 / 1.81	1.77 / 1.78	1.86 / 1.86	1.79 / 1.77	1.45 / 1.46
qwen3.5	1.92 / 1.91	1.83 / 1.83	1.89 / 1.88	1.85 / 1.82	1.47 / 1.49
mistral-tekken	1.92 / 1.92	1.84 / 1.84	1.96 / 1.96	1.85 / 1.84	1.48 / 1.50
apertus	1.92 / 1.92	1.84 / 1.84	1.96 / 1.96	1.85 / 1.84	1.48 / 1.50
llama3	2.03 / 2.00	1.91 / 1.93	2.04 / 2.03	1.93 / 1.91	1.54 / 1.55
sailor2	2.03 / 2.04	1.92 / 1.93	2.03 / 2.04	1.94 / 1.93	1.54 / 1.56

Table 3: Words split into ≥3 tokens / UTF-8 bytes per token. Source: `fragmentation_rate_ge3` / `compression_bytes_per_token`.

tokenizer	eng	fil	ceb	ilo	war	pag
gemma4	0.044 / 4.87	0.248 / 3.38	0.246 / 3.31	0.275 / 3.17	0.220 / 3.40	0.233 / 3.31
qwen3.5	0.045 / 4.80	0.291 / 3.15	0.277 / 3.15	0.304 / 3.08	0.247 / 3.24	0.254 / 3.20
mistral-tekken	0.050 / 4.75	0.295 / 3.12	0.285 / 3.10	0.313 / 2.93	0.251 / 3.21	0.265 / 3.15
apertus	0.050 / 4.75	0.295 / 3.12	0.285 / 3.10	0.313 / 2.93	0.251 / 3.21	0.265 / 3.15
llama3	0.043 / 4.86	0.316 / 3.02	0.295 / 3.06	0.327 / 2.89	0.266 / 3.14	0.280 / 3.10
sailor2	0.047 / 4.78	0.328 / 2.96	0.306 / 3.00	0.334 / 2.85	0.275 / 3.08	0.292 / 3.05

Table 4: Native text (Wikipedia) and Taglish, tokens per word. Absolute values: these sets are not parallel.

tokenizer	tl	ceb	ilo	war	Taglish PH-EVAL (n=60)	Taglish FiReCS (n=1,000)
gemma4	1.946	2.205	1.949	1.813	1.466	1.312
qwen3.5	2.071	2.339	2.030	2.207	1.594	1.428
mistral-tekken	2.065	2.381	2.107	2.172	1.599	1.464
apertus	2.065	2.381	2.107	2.172	1.599	1.464
llama3	2.128	2.366	2.128	2.229	1.626	1.462
sailor2	2.194	2.487	2.182	2.317	1.639	1.484

Hypothesis test. Filipino tokens per word: gemma4 1.840 < qwen3.5 1.974 < sailor2 2.098 (Table 1). **H-DEC-001 Part B is supported.** Gemma 4 uses 6.8% fewer tokens than Qwen3.5 and 12.3% fewer than Sailor2 (computed as 1 − 1.840/1.974 and 1 − 1.840/2.098).

Checks. The same-family tokenizers are byte-identical (length plus vocab sha256): Qwen3.5 9B = 4B, and Gemma 4 12B = E4B. **Reproduction:** a second team member reran the pipeline in an independent environment and compared 876 values: 0 differences.

5. Discussion

The inflation is large and universal. Roughly 25–33% of Filipino words split into three or more tokens, against 4–5% of English words (Table 3). That fits the affixation and reduplication that TOKENIZER_RESEARCH.md anticipated. It is not yet traced to specific affixes; that needs the probe word lists.

The differences between candidates are small next to the shared gap. The best tokenizer (Gemma 4, 262K vocabulary) still costs 1.8× English on Filipino.

Two results were unexpected:

- **Sailor2 is the least efficient** tokenizer, even though its pretraining explicitly covers Tagalog, Cebuano, Ilocano and Waray. It kept Qwen2.5′s vocabulary unchanged. So regional pretraining does not by itself buy tokenizer efficiency.
- **Taglish is cheaper than pure Filipino** (1.31–1.64 tokens per word), because its English words tokenize well.

For PinoAI this has three consequences:

1. Continued pretraining on Philippine text will cost about 1.8–2× the tokens of the same volume of English. Compute plans should budget for that.
2. Tokenizer efficiency favors Gemma 4, but only modestly. It should not decide DEC-001 on its own; Part A (PH-EVAL quality) matters more.
3. Vocabulary extension is the only lever that could close most of the gap. It changes embeddings, so it must earn its cost in a controlled experiment (EXP-TOK-002).

6. Limitations and threats to validity

- FLORES+ is translated from English. Native Filipino with heavier affixation may inflate more than measured here.
- **Hiligaynon is not measured:** no text source exists yet (no FLORES+, no Wikipedia, 44 Tatoeba sentences).
- The Cebuano Wikipedia sample comes from shard 0 of 10, and Cebuano Wikipedia is reported to be largely bot-generated.
- The PH-EVAL Taglish set is small (n = 60) and skews toward tech/ML contexts.
- The Llama-3-style row is SmolLM3′s tokenizer. It was not compared with Meta's gated original.
- Tokens per word depends on whitespace segmentation. Inflation on aligned sentences does not, and is the primary measure.

7. Next steps

- DEC-001: keep the original tokenizer. Await Part A.
- EXP-TOK-002: extend the chosen model's vocabulary with PH-Corpus subwords, then re-measure and run a short continued pretraining. Train on corpus text only, never on eval sources.
- Addenda: a Hiligaynon measurement (once a source exists), and affix probe lists with a fragmentation heatmap (after native review).

Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

AI assistance. Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.

References

[1] P. Rust and others, "How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models," *arXiv preprint arXiv:2012.15613*, 2020.

[2] A. Petrov and others, "Language Model Tokenizers Introduce Unfairness Between Languages," *arXiv preprint arXiv:2305.15425*, 2023.

[3] NLLB Team, "No Language Left Behind: Scaling Human-Centered Machine Translation," *arXiv preprint arXiv:2207.04672*, 2022.